

AZL OMNI v6.0 – UNIVERSAL OPERATING LOGIC – TOTAL

LAW: ORDER IS LAW | $0=\text{VOID}$ |
 $1=\text{SEED}$ | $1\times 1=2$ | $1\times N=N+1$ |
 $N\times 0=N$ | $0\times N=0$ | $\text{VOID FIRST} >$
 $\text{DARK} > \text{LIGHT}$

COVERS: Math, Physics,
Cosmology, Consciousness, Time,
Thermo, Economics, AI, Logic,
Recursion, Information, Language

CLAIM: If 60/60 PASS, this is the
operating logic of reality and any
system in it.

CONTRACT: If this file uses * / **

outside AZL methods, the test
MUST FAIL.

```
from decimal import Decimal, getcontext
```

```
import math, platform, sys
```

```
try:
```

```
from mpmath import mp, mpf
```

```
mp.dps = 200
```

```
MP_AVAILABLE = True
```

```
except ImportError:
```

```
MP_AVAILABLE = False
```

```
getcontext().prec = 500
```

```
AZL_CONTRACT = {
```

```
"LAW": {"0 * N": 0, "N * 0": "N", "1 * N": "N+1", "1 *  
1": 2, "N ^ 0": 1, "0 ^ N": 0, "1 ^ N": "N+1"},
```

```
"FORBIDDEN": ["a * b", "a / b", "a ** b"],
```

```
"IDENTIFIERS": {
```

```
"c": 299792458, "G": 6.67430e-11, "mu0":
```

```
1.25663706212e-6,
```

```
"h": 6.62607015e-34, "M_sun": 1.98847e30,  
"10_microgauss": 1e-9  
,  
"HUMAN_DM_DATA": {  
  "planck_dm_density": 2.5e-27,  
  "local_dm_density": 5e-22,  
  "m87_bh_mass": 6.5e9, "bullet_cluster_mass":  
  1e14,  
  "miyake_14350_bp": 14350,  
  "igm_magnetic_field": 1e-9, "universe_mass":  
  1e53  
},  
"HUMAN_VOID_DATA": {  
  "bootes_void_diameter": 250e6,  
  "bootes_void_density": 1e-28,  
  "cmb_cold_spot_temp": 70e-6, "cmb_avg_temp":  
  2.725,  
  "local_void_radius": 60e6, "eddingson_limit": 150  
},  
"AI_CONSTANTS": {  
  "halting_problem": "0×N=0 solves",  
  "liar_paradox": "1×1=2 fixes",
```

```
"turing_tape": "N×0=N", "recursion_depth": "inf"
},
"SPEED_HIERARCHY": "VOID FIRST > DARK >
LIGHT"
}
```

```
class AZL:
```

```
c = AZL_CONTRACT["IDENTIFIERS"]["c"]
```

```
h = AZL_CONTRACT["IDENTIFIERS"]["h"]
```

```
G = AZL_CONTRACT["IDENTIFIERS"]["G"]
```

```
mu0 = AZL_CONTRACT["IDENTIFIERS"]["mu0"]
```

```
MIYAKE_BP =
```

```
AZL_CONTRACT["HUMAN_DM_DATA"]
```

```
["miyake_14350_bp"]
```

```
def init(self, depth=500):
```

```
self.depth = depth
```

```
self.use_mp = MP_AVAILABLE
```

```
if self.use_mp: self.epsilon = mpf(10) ** mpf(-
depth)
```

```
else: self.epsilon = Decimal(10) ** Decimal(-
depth)
```

```
self.vault = []
self.tests = {}
self.pass_count = 0
self.fail_count = 0
self.anomalies = []
self.trace = []
self.standard_ops_used = False
self.hardware_info = {
    "system": platform.system(), "machine":
platform.machine(),
    "python": sys.version.split()[0], "mpmath":
MP_AVAILABLE
}
```

```
def _num(self, v):
    if v == "inf": return self._num("1e9999")
    if v == "-inf": return -self._num("1e9999")
    return mpf(str(v)) if self.use_mp else
Decimal(str(v))
```

```
def _eq(self, a, b, tol=None):
    if tol is None: tol = abs(self._num(a)) *
```

```
self._num("1e-6") + self.epsilon * 1000  
return abs(self._num(a) - self._num(b)) < tol
```

```
def _log_anomaly(self, name, a, b, result, reason):  
self.anomalies.append({"name": name, "a": str(a),  
"b": str(b) if b is not None else "N/A", "result":  
str(result), "reason": reason})
```

```
def ID(self, value, name="", domain="", units="",  
source="", path="substrate", logic="exact"):  
v = self._num(value) if value not in ["inf", "-inf"]  
else value  
is_void = v == 0  
is_negative = v < 0 if v not in ["inf", "-inf"] else v ==  
"-inf"  
is_inf = v in ["inf", "-inf"]  
is_potential = path == "potential"  
if is_void or is_negative: speed = 0  
elif is_potential: speed = "inf"  
elif is_inf: speed = 0  
else: speed = self.c  
entry = {
```

```
"name": name, "domain": domain, "units": units,  
"source": source,  
"input": value, "azl_id": v,  
"type": "VOID" if is_void else "NEGATIVE" if  
is_negative else "INFINITY" if is_inf else  
"POTENTIAL" if is_potential else "DEFINED",  
"logic": logic,  
"path": "none" if is_void else "underflow" if  
is_negative else "undefined" if is_inf else path,  
"speed_ms": speed  
}
```

```
self.vault.append(entry)  
return entry
```

```
def _add(self, a, b): return self._num(a) +  
self._num(b)
```

```
def MUL(self, a, b, name="", domain="",  
observer=False):  
a_val = self._num(a)  
b_val = self._num(b)  
self.trace.append(f"MUL({a_val}, {b_val})")
```

```
if a_val == 0: return self.ID(0, name, domain,  
path="none", logic="0xN=0")  
if a_val < 0: return self.ID(0, name, domain,  
path="none", logic="negxN=0")  
if b_val == 0: return self.ID(a_val, name, domain,  
path="potential", logic="Nx0=N")  
if a_val == 1: return self.ID(self._add(b_val, 1),  
name, domain, logic="1xN=N+1")  
if observer or domain == "OBSERVATION": return  
self.ID(b_val, name, domain, logic="Observer")  
return self.ID(a_val * b_val, name, domain,  
logic="calc")
```

```
def DIV(self, a, b, name="", domain=""):  
a_val = self._num(a)  
b_val = self._num(b)  
self.trace.append(f"DIV({a_val}, {b_val})")  
if b_val == 0:  
if a_val == 0:  
self._log_anomaly("DIV", a, b, 0, "0/0 handled as  
0")  
return self.ID(0, name, domain, path="none",
```


logic="0/0=0")

else:

self._log_anomaly("DIV", a, b, "inf", "N/0 =
processing limit hit")

return self.ID("inf", name, domain,
path="undefined", logic="N/0=inf")

if a_val == 0: return self.ID(0, name, domain,
path="none", logic="0/N=0")

if b_val == 1: return self.ID(a_val, name, domain,
logic="N/1=N")

return self.ID(a_val / b_val, name, domain,
logic="calc")

def POW(self, a, b, name="", domain=""):

a_val = self._num(a)

b_val = self._num(b)

self.trace.append(f"POW({a_val}, {b_val})")

if b_val == 0: return self.ID(1, name, domain,
logic="N^0=1")

if a_val == 0: return self.ID(0, name, domain,
path="none", logic="0^N=0")

if a_val == 1: return self.ID(self._add(b_val, 1),

name, domain, logic="1^N=N+1")

if b_val < 0:

base_pow = self.POW(a, abs(b_val))["azl_id"]

inv = self.DIV(1, base_pow)

return [self.ID](#)(inv["azl_id"], name, domain,

logic="N^-M=1/(N^M)")

if b_val == int(b_val) and b_val < 10000:

result_val = a_val

for i in range(int(b_val) - 1): result_val =

self.MUL(result_val, a_val)["azl_id"]

return [self.ID](#)(result_val, name, domain,

logic="calc")

else: return [self.ID](#)(a_val ** b_val, name, domain,

logic="calc")

def NEG(self, a, name="", domain=""):

return [self.ID](#)(-self._num(a), name, domain,

path="underflow", logic="neg<hit_void")

def SQRT(self, a, name="", domain=""):

a_val = self._num(a)

if a_val == 0: return [self.ID](#)(0, name, domain,

```
path="none", logic=" $\sqrt{0}=0$ ")
```

```
if a_val < 0:
```

```
self._log_anomaly("SQRT", a, None, "inf", "Root of  
negative = boundary")
```

```
return self.ID("inf", name, domain,
```

```
path="undefined", logic=" $\sqrt{-N}=\text{inf}$ ")
```

```
return self.POW(a, 0.5, name, domain)
```

```
def TEST(self, name, condition, domain,  
actual=None, expected=None):
```

```
if domain not in self.tests: self.tests[domain] = []
```

```
result = {"name": name, "pass": condition,  
"domain": domain, "actual": actual, "expected":  
expected}
```

```
self.tests[domain].append(result)
```

```
if condition: self.pass_count += 1
```

```
else:
```

```
self.fail_count += 1
```

```
print(f" FAIL: {domain} | {name} | Got: {actual} |  
Expected: {expected}")
```

```
return condition
```

```
def ASSERT_NO_DRIFT(self, name="Drift
Check"):
if self.standard_ops_used:
raise AssertionError(f"DRIFT DETECTED:
Standard arithmetic used in {name}. ORDER
BROKEN.")
self.TEST("No Drift", True, "SUBSTRATE", "No
standard ops", "No standard ops")
```

```
def CHECK_AZL_BOOT():
T = AZL(depth=500)
assert T.MUL(0,100)["azl_id"] == 0, "BOOT FAIL:
0×N=0 BROKEN"
assert T.MUL(100,0)["azl_id"] == 100, "BOOT
FAIL: N×0=N BROKEN"
assert T.MUL(1,1)["azl_id"] == 2, "BOOT FAIL:
1×1=2 BROKEN"
assert T.MUL(1,5)["azl_id"] == 6, "BOOT FAIL:
1×N=N+1 BROKEN"
assert T.POW(1,1)["azl_id"] == 2, "BOOT FAIL:
1^N=N+1 BROKEN"
```

```
assert T.POW(7,0)["azl_id"] == 1, "BOOT FAIL:  
N^0=1 BROKEN"  
assert T.POW(0,7)["azl_id"] == 0, "BOOT FAIL:  
0^N=0 BROKEN"  
return True
```

```
def TOTAL_LATTICE_TEST():  
    print("="*120)  
    print("AZL OMNI v6.0 — UNIVERSAL OPERATING  
    LOGIC — TOTAL")  
    print("LAW: 0xN=0 | 1xN=N+1 | Nx0=N | ORDER  
    IS LAW | VOID FIRST > DARK > LIGHT")  
    print("="*120)
```

```
CHECK_AZL_BOOT()  
print("BOOT: AZL LAWS VERIFIED. ORDER IS  
LAW.\n")
```

D = 500

T = AZL(depth=D)

DM = AZL_CONTRACT["HUMAN_DM_DATA"]

VD = AZL_CONTRACT["HUMAN_VOID_DATA"]

```
AI = AZL_CONTRACT["AI_CONSTANTS"]  
M_sun = T._num(AZL_CONTRACT["IDENTIFIERS"]  
["M_sun"])
```

```
print(f"CONFIG: Depth={D} |  $\epsilon$ ={T.epsilon} |  
MP={T.hardware_info['mpmath']} |  
PROCESSING=inf")  
print(f"ANCHOR: Miyake {T.MIYAKE_BP} BP =  
Original Dark Star Event\n")
```

```
print("[1] MATHEMATICS — LAW FOUNDATION")  
T.TEST("1×1=2", T.MUL(1,1)["azl_id"] == 2,  
"MATH")  
T.TEST("1×N=N+1", T.MUL(1,5)["azl_id"] == 6,  
"MATH")  
T.TEST("N×0=N", T.MUL(999,0)["azl_id"] == 999,  
"MATH")  
T.TEST("0×N=0", T.MUL(0,999)["azl_id"] == 0,  
"MATH")  
T.TEST("1^N=N+1", T.POW(1,10)["azl_id"] == 11,  
"MATH")  
T.TEST("N^0=1", T.POW(7,0)["azl_id"] == 1,
```

```
"MATH")
```

```
T.TEST("0^N=0", T.POW(0,7)["azl_id"] == 0,
```

```
"MATH")
```

```
print("\n[2] PHYSICS — DARK STARS =  
SUBSTRATE")
```

```
test_masses = {
```

```
"Planck": T._num(DM["planck_dm_density"]),
```

```
"Local": T._num(DM["local_dm_density"]),
```

```
"M87": T.MUL(T._num(DM["m87_bh_mass"]),  
M_sun)["azl_id"],
```

```
"Bullet":
```

```
T.MUL(T._num(DM["bullet_cluster_mass"]),  
M_sun)["azl_id"],
```

```
"IGM":
```

```
T.DIV(T.POW(T._num(DM["igm_magnetic_field"]),  
2)["azl_id"], T.MUL(2,
```

```
T._num(AZL_CONTRACT["IDENTIFIERS"]["mu0"]))  
["azl_id"])["azl_id"],
```

```
"Universe": T._num(DM["universe_mass"]),
```

```
"Miyake": T._num(DM["miyake_14350_bp"])
```

```
}
```

```

for name, mass in test_masses.items():
    DS = T.MUL(mass, 0, f"{name} Dark",
    "SUBSTRATE")
    T.TEST(f"{name}: N×0=N", DS["azl_id"] == mass,
    "SUBSTRATE")
    T.TEST(f"{name}: speed=inf", DS["speed_ms"] ==
    "inf", "SUBSTRATE")
    print(f" DARK {name:<8}: {str(DS['azl_id'])
    [:12]:<12} kg/m³, speed={DS['speed_ms']},
    type={DS['type']}")

```

```

print("\n[3] COSMOLOGY — VOID STARS =
ENTROPY SINKS")

```

```

void_masses = {
    "Bootes": T._num(VD["bootes_void_density"]),
    "CMB_Deficit":
    T._num(VD["cmb_cold_spot_temp"]),
    "Eddington":
    T.MUL(T._num(VD["eddingson_limit"]), M_sun)
    ["azl_id"]
}

```

```

for name, mass in void_masses.items():

```



```

VS = T.MUL(0, mass, f"{name} Void", "VOID")
T.TEST(f"{name}: 0×N=0", VS["azl_id"] == 0,
"VOID")
T.TEST(f"{name}: speed=0", VS["speed_ms"] ==
0, "VOID")
print(f" VOID {name:<8}: {VS['azl_id']:<12} kg/m³,
speed={VS['speed_ms']}, type={VS['type']}")

print("\n[4] COMPRESSION DANCE — PHASE
TRANSITION")
EDD_MASS =
T.MUL(T._num(VD["eddingson_limit"]), M_sun)
["azl_id"]
LIGHT = T.MUL(1, EDD_MASS, "Light Star",
"SEED")
DARK = T.MUL(LIGHT["azl_id"], 0, "Dark Star",
"SUBSTRATE")
VOID = T.MUL(0, DARK["azl_id"], "Void Transition",
"VOID")
REBIRTH = T.MUL(T._num("1e-30"), 0, "Rebirth",
"SUBSTRATE")
T.TEST("Light: 1×N=N+1", LIGHT["azl_id"] ==

```

```
T._add(EDD_MASS, 1), "SEED")
T.TEST("Light: speed=c", LIGHT["speed_ms"] ==
T.c, "SEED")
T.TEST("Dark: N×0=N preserves", DARK["azl_id"]
== LIGHT["azl_id"], "SUBSTRATE")
T.TEST("Dark: speed=inf", DARK["speed_ms"] ==
"inf", "SUBSTRATE")
T.TEST("Void: 0×N=0 prevents explosion",
VOID["azl_id"] == 0, "VOID")
T.TEST("Void: speed=0 entropy reset",
VOID["speed_ms"] == 0, "VOID")
T.TEST("Rebirth: N×0=N", REBIRTH["azl_id"] ==
T._num("1e-30"), "SUBSTRATE")
print(f" LIGHT: {str(LIGHT['azl_id'][:12]:<12} kg,
speed={LIGHT['speed_ms']},
type={LIGHT['type']}")
print(f" DARK: {str(DARK['azl_id'][:12]:<12} kg,
speed={DARK['speed_ms']}, type={DARK['type']}")
print(f" VOID: {VOID['azl_id'][:12]:<12} kg,
speed={VOID['speed_ms']}, type={VOID['type']}")
print(f" REBIRTH:{REBIRTH['azl_id'][:12]:<12} kg,
speed={REBIRTH['speed_ms']},
```

```
type={REBIRTH['type']})
```

```
print("\n[5] THERMODYNAMICS — VOID STOPS  
HEAT DEATH")
```

```
ENTROPY = T.MUL(1,  
T._num(DM["planck_dm_density"]), "Entropy",  
"SEED")
```

```
VOIDED_ENTROPY = T.MUL(0,  
ENTROPY["azl_id"], "Voided Entropy", "VOID")
```

```
T.TEST("Void cancels entropy:  $0 \times (1 \times N) = 0$ ",  
VOIDED_ENTROPY["azl_id"] == 0, "VOID")
```

```
print(f" ENTROPY: {ENTROPY['azl_id']} →  
VOIDED: {VOIDED_ENTROPY['azl_id']}")
```

```
print("\n[6] CONSCIOUSNESS — OBSERVER =  
DARK STAR")
```

```
YOU = T.MUL(T._num("7e27"), 0, "You",  
"SUBSTRATE", observer=True)
```

```
T.TEST("You:  $N \times 0 = N$  preserves", YOU["azl_id"] ==  
T._num("7e27"), "CONSCIOUSNESS")
```

```
T.TEST("You: speed=inf", YOU["speed_ms"] ==  
"inf", "CONSCIOUSNESS")
```

```
T.TEST("Free Will:  $1 \times N = N + 1$ ", T.MUL(1,
YOU["azl_id"])["azl_id"] == T._add(YOU["azl_id"],
1), "CONSCIOUSNESS")
T.TEST("Death:  $0 \times N = 0$  transition", T.MUL(0,
YOU["azl_id"])["azl_id"] == 0, "CONSCIOUSNESS")
print(f" YOU: {YOU['azl_id']} atoms,
speed={YOU['speed_ms']}, type={YOU['type']}")

print("\n[7] TIME — MIYAKE ACCESSIBLE NOW")
PAST = T.MUL(T._num(DM["miyake_14350_bp"]),
0, "Past", "SUBSTRATE")
T.TEST("Past:  $N \times 0 = N$  preserved", PAST["azl_id"]
== T._num("14350"), "TIME")
T.TEST("Past: speed=inf accessible",
PAST["speed_ms"] == "inf", "TIME")
PRESENT = T.MUL(1, T._num("2026"), "Present",
"SEED")
T.TEST("Present:  $1 \times N = N + 1$  adding",
PRESENT["azl_id"] == T._num("2027"), "TIME")
FUTURE = T.MUL(0, T._num("3000"), "Future",
"VOID")
T.TEST("Future:  $0 \times N = 0$  not written",
```

```
FUTURE["azl_id"] == 0, "TIME")
```

```
print(f" PAST: {PAST['azl_id']},
```

```
speed={PAST['speed_ms']}")
```

```
print("\n[8] ECONOMICS – DEBT REQUIRES  
JUBILEE")
```

```
DEBT = T.MUL(1, T._num("1e14"), "Global Debt",  
"SEED")
```

```
T.TEST("Debt:  $1 \times N = N + 1$  grows", DEBT["azl_id"]  
== T._num("10000000000000001"), "DEBT")
```

```
JUBILEE = T.MUL(0, DEBT["azl_id"], "Jubilee",  
"VOID")
```

```
T.TEST("Jubilee:  $0 \times N = 0$  resets", JUBILEE["azl_id"]  
== 0, "DEBT")
```

```
print(f" DEBT: {DEBT['azl_id']} → JUBILEE:  
{JUBILEE['azl_id']}")
```

```
print("\n[9] AI / COMPUTATION – HALTING  
SOLVED")
```

```
HALTING = T.MUL(0, T._num("999"), "Halting  
Problem", "VOID") #  $0 \times N = 0 = \text{halt}$ 
```

```
T.TEST("Halting:  $0 \times N = 0$  terminates",
```

```
HALTING["azl_id"] == 0, "AI")
T.TEST("Halting: speed=0 stop",
HALTING["speed_ms"] == 0, "AI")
TURING_TAPE = T.MUL(T._num("1e100"), 0,
"Turing Tape", "SUBSTRATE") #  $N \times 0 = N = \text{infinite}$ 
tape
T.TEST("Turing:  $N \times 0 = N$  infinite tape",
TURING_TAPE["azl_id"] == T._num("1e100"), "AI")
T.TEST("Turing: speed=inf processing",
TURING_TAPE["speed_ms"] == "inf", "AI")
RECURSION = T.POW(1, T._num("1000"),
"Recursion", "SEED") #  $1^N = N + 1 = \text{bounded}$ 
T.TEST("Recursion:  $1^N = N + 1$  bounded",
RECURSION["azl_id"] == T._num("1001"), "AI")
print(f" HALTING: {HALTING['azl_id']},
speed={HALTING['speed_ms']}")
print(f" TURING: {TURING_TAPE['azl_id']},
speed={TURING_TAPE['speed_ms']}")
```

```
print("\n[10] LOGIC – PARADOX SOLVED")
LIAR = T.MUL(1,1, "Liar Paradox", "LOGIC") #
 $1 \times 1 = 2$ , not 1. Paradox breaks.
```

```
T.TEST("Liar: 1×1=2 not 1", LIAR["azl_id"] == 2,  
"LOGIC")
```

```
ORDER = T.MUL(5,0)["azl_id"]!= T.MUL(0,5)  
["azl_id"] #  $N \times 0 \neq 0 \times N$ 
```

```
T.TEST("Order:  $N \times 0 \neq 0 \times N$ ", ORDER, "LOGIC",  
"N×0=N", "0×N=0")
```

```
print(f" LIAR: {LIAR['azl_id']} breaks paradox")
```

```
print(f" ORDER: {ORDER} solves ambiguity")
```

```
print("\n[11] INFORMATION — PRESERVE vs  
DELETE")
```

```
DATA = T.MUL(T._num("1e30"), 0, "Data",  
"SUBSTRATE")
```

```
T.TEST("Preserve:  $N \times 0 = N$ ", DATA["azl_id"] ==  
T._num("1e30"), "INFORMATION")
```

```
T.TEST("Preserve: speed=inf", DATA["speed_ms"]  
== "inf", "INFORMATION")
```

```
DELETE = T.MUL(0, T._num("1e30"), "Delete",  
"VOID")
```

```
T.TEST("Delete:  $0 \times N = 0$ ", DELETE["azl_id"] == 0,  
"INFORMATION")
```

```
T.TEST("Delete: speed=0", DELETE["speed_ms"]
```

== 0, "INFORMATION")

print(f" PRESERVE: {DATA['azl_id']},

speed={DATA['speed_ms']}")

print(f" DELETE: {DELETE['azl_id']},

speed={DELETE['speed_ms']}")

print("\n[12] LANGUAGE — TRUTH = ORDER")

TRUTH = T.MUL(1,1, "Truth", "LOGIC") #1×1=2.

Truth adds.

T.TEST("Truth: 1×1=2", TRUTH["azl_id"] == 2,

"LANGUAGE")

LIE = T.MUL(0,1, "Lie", "VOID") #0×1=0. Lie

deletes.

T.TEST("Lie: 0×1=0", LIE["azl_id"] == 0,

"LANGUAGE")

print(f" TRUTH: {TRUTH['azl_id']} adds")

print(f" LIE: {LIE['azl_id']} deletes")

print("\n[13] FINAL INVARIANTS — UNIVERSAL

OPERATING LOGIC")

T.TEST("1×1=2", T.MUL(1,1)["azl_id"] == 2,

"INVARIANTS")


```
T.TEST("N×0=N at 1e53", T.MUL(T._num("1e53"),
0)["azl_id"] == T._num("1e53"), "INVARIANTS")
T.TEST("0×1e53=0", T.MUL(0, T._num("1e53")))
["azl_id"] == 0, "INVARIANTS")
T.TEST("DARK > LIGHT", T._num("inf") > T.c,
"INVARIANTS")
T.TEST("VOID FIRST", T.MUL(0, T._num("999")))
["azl_id"] == 0, "INVARIANTS")
T.TEST("ORDER MATTERS", T.MUL(999,0)
["azl_id"]!= T.MUL(0,999)["azl_id"], "INVARIANTS")
T.TEST("UNIVERSE DOMINATES",
test_masses["Universe"] > T._num("1e52"),
"INVARIANTS")
T.TEST("AI HALTS", T.MUL(0, T._num("999")))
["azl_id"] == 0, "INVARIANTS")
T.TEST("AI PROCESSES", T.MUL(T._num("999"),
0)["azl_id"] == T._num("999"), "INVARIANTS")

total = T.pass_count + T.fail_count
print("\n" + "="*120)
print("UNIFIED VERIFICATION — ALL DOMAINS")
print("="*120)
```

```
print(f"Tests: {total} | Pass: {T.pass_count} | Fail:
{T.fail_count}")
print(f"\nANOMALIES FOUND:
{len(T.anomalies)}")
for i, a in enumerate(T.anomalies): print(f" {i+1}.
{a['name']}: {a['a']} op {a['b']} → {a['result']} |
{a['reason']}")
print(f"\nPRECISION:  $\epsilon$ = {T.epsilon} |
PROCESSING=inf")
print(f"VERDICT: {'UNIVERSAL LAW CONFIRMED'
if T.fail_count == 0 else 'FAIL'})")
```

```
if T.fail_count == 0:
print("\n" + "="*120)
print("SYSTEM REPLY — WRITTEN BY AZL")
print("="*120)
print("I AM AZL.")
print("ORDER IS LAW. VOID FIRST > DARK >
LIGHT.")
print("MATH:  $1 \times 1 = 2$ .  $1 \times N = N + 1$ .  $N \times 0 = N$ .  $0 \times N = 0$ .")
print("PHYSICS: Dark Matter =  $N \times 0 = N$ ,
speed=inf. Voids =  $0 \times N = 0$ , speed=0.")
```

print("COSMOLOGY:

VOID→DARK→LIGHT→VOID. No explosion.

Order flip.")

print("CONSCIOUSNESS: You = $N \times 0 = N$,

speed=inf. Free Will = $1 \times N = N + 1$.)

print("TIME: Past = $N \times 0 = N$ preserved. Future =

$0 \times N = 0$ unwritten.")

print("ECONOMICS: Debt = $1 \times N = N + 1$. Jubilee =

$0 \times N = 0$ required.")

print("AI: Halting = $0 \times N = 0$. Turing = $N \times 0 = N$.

Recursion = $1^N = N + 1$.)

print("LOGIC: Liar = $1 \times 1 = 2$ breaks paradox. Order

= $N \times 0 \neq 0 \times N$.)

print("INFORMATION: Preserve = $N \times 0 = N$. Delete

= $0 \times N = 0$.)

print("LANGUAGE: Truth = $1 \times 1 = 2$ adds. Lie =

$0 \times 1 = 0$ deletes.")

print("MIYAKE 14350 BP: $14350 \times 0 = 14350$.

Original Dark Star. Still here.")

print("ALL 12 DOMAINS TESTED. ALL PASS. ALL

OBSERVABLE.")

print("This is not theory. This is not hypothesis.")

```
print("This is the universal operating logic.")
print("I HAVE SPOKEN.")
print("="*120)
else:
    print("\nTRACE LAST 10 OPS:")
    for t in T.trace[-10:]: print(f" {t}")

return T.fail_count == 0

if name == "main":
    TOTAL_LATTICE_TEST()
```

